JPA, Part 4

JPQL, Criteria API

Learning goals

1. Learn to perform mass queries and update operations with JPQL.
2. Learn the fundamentals of programmatically constructed Criteria queries.

Foundations

- The idea of JPA is to hide the relational database from the software developer.
- The persistence and retrieval of objects is normally done with `EntityManager`'s `persist()` and `find()` methods.
  - In Spring boot, use them via `save()` and `findByid()`, provided by the repository.
- Some operations cannot be efficiently implemented with these operations.
- Examples of 'difficult' operations:
  - Retrieve all employees earning €2000/mo. or more.
  - Grant a 5% pay-rise to all employees.
- Retrieval and persistence of all relevant objects one at a time would be very inefficient.
  - The solution would generate a huge number of SQL statements, even though one statement would suffice for each task.

Solution options

1. JPQL (*Jakarta Persistence Query Language*)
   - An object-oriented query language.
   - Hibernate transforms the JPQL statements into SQL statements and packs the result set into Java data structures.
2. SQL
   - Hibernate provides a possibility to relay SQL queries to the DBMS. Again, the results are fetched into Java data structures.
3. Criteria API
   - Provides classed and methods that allow specifying the desired operation without a dedicated query language.

- Next, the main focus will be on JPQL.

Example: retrieval of object with search criteria

```
SELECT t
FROM Tili t
WHERE saldo>=3000
```

```
SELECT h
FROM Haltija h
WHERE h.etunimi LIKE "M%"
```

- Fetch all employees earning at least €3000/mo. into a list structure.
- The query produces Employee objects.
  - On the left, an alias (named e) is required for expressing the query.

Spring Boot: How to define the query

1.

```
@GetMapping("/etunimi/{alku}")  vesavvo
public ResponseEntity<List<HaltijaDto>> getHaltijatEtunimiAlkaa(@PathVariable String alku) {
    List<HaltijaDto> haltijat = haltijaService.haeHaltijatJoidenNimiAlkaa(alku);
    return ResponseEntity.ok(haltijat);
}
```

2.

```
public interface HaltijaRepository extends JpaRepository<Haltija, Integer>

    @Query("SELECT h FROM Haltija h WHERE h.etunimi LIKE :alku%")  1 usage
    List<Haltija> findByEtunimiAlkaa(@Param("alku") String alku);

}
```

3..

```
public List<HaltijaDto> haeHaltijatJoidenNimiAlkaa(String alku) {  1 usage  vesavvo

    List<Haltija> haltijat = haltijaRepository.findByEtunimiAlkaa(alku);

    List<HaltijaDto> dtot = new ArrayList<>();
    for (Haltija h : haltijat) {
        HaltijaDto dto = new HaltijaDto(h.getId(), h.getEtunimi(), h.getSukunimi());
        dtot.add(dto);
    }
    return dtot;
}
```

1. Add the endpoint in the controller
2. Define the query in the repository
3. Call the repository method in the service

Data manipulation queries

- As an object's data changes, it is sufficient to change the instance variable values.
- If the object is managed by `EntityManager`, the changes are reflected in the database at the transaction commit.
  - An object is managed by `EntityManager`, when it has been subject to a `find()` or a `persist()/merge()` operation, or it has been retrieved as an entity via a JPQL or Criteria query in the same persistence context.
    - Spring Data JPA repository methods `findById()` and `save()` typically delegato te these `EntityManager` objects.
  - An individual object is deleted from the database by calling `EntityManager.remove()`.
    - Repository method `deleteById()` typically performs the removal via this mechanism.
- Update queries are used for batch updates or deletes.

Data manipulation queries

- JPQL provides means for inserting, deleting and updating objects.
- This results a single SQL operation, thus eliminating the need to do the modifications one record at a time with many SQL statements.
- Operations:
  - UPDATE
  - DELETE
  - INSERT
- These operations generate the corresponding SQL statements.

Example: an update query

```
UPDATE Tili
SET saldo=saldo*kerroin
```

```java
public interface TiliRepository extends JpaRepository<Tili, Integer> {

    @Modifying  1 usage  new *
    @Query("UPDATE Tili t SET t.saldo = t.saldo * :kerroin")
    int kasvataKaikkienSaldoja(@Param("kerroin") double kerroin);
```

- The query raises each employee's salary by a given percentage.
- The number of affected objects is returned.

On using SQL

- In some situations, one may prefer SQL to JPQL:
  - The earlier implementation may provide working and tested SQL statements.
  - A query may be very complex and require optimization. It may be difficult to express it in JPQL in such a way that the desired SQL statement is generated.
  - The developer may have good command of SQL but not enough resources to learn JPQL.
- For relaying SQL operations, one can use `EntityManager`'s `createNativeQuery()` method.
- In Spring Boot, it is enough to add the `nativeQuery=true` modifier into the `@Query` annotation in the repository.

N+1 problem

- N+1 problem is a well-know performance problem with JPA.
- It refers to the situation where eager loading of 1:M associated entities produce a large number of DB queries as a chain reaction.
- This may produce a significant performance bottleneck.
- The problem arises with sloppily written batch SELECTs in JPQL queries.

N+1 problem

Employee.java

```
@OneToMany(fetch=FetchType.EAGER)
@JoinColumn
private List<Computer> computers = new ArrayList<Computer>();
```

Computer.java

```
@Id
private String serialNumber;
private String brand;
```



- This time, consider a 1:M association between Employee and Computer.
  - 1:M, eager loading defined for ’1’ side: fetch=FETCHTYPE.EAGER
- The problem arises when a repository’s JPQL query retrieves all employees:
  - FROM Employee;

N+1 problem

```
Hibernate: select employee0_.empId as empId1_2_, employee0_.department_deptId as departme5_2_,
```

1

```
Hibernate: select computers0_.computers_empId as computer3_0_0_, computers0_.serialNumber as se
Hibernate: select computers0_.computers_empId as computer3_0_0_, computers0_.serialNumber as se
Hibernate: select computers0_.computers_empId as computer3_0_0_, computers0_.serialNumber as se
```

N

- The query results in an inefficient implementation where
  - Employees are queried with one SELECT.
  - For each employee, a query for his/her computers is generated. That query only retrieves the computers of one employee.
  - If there are N employees, the result is N+1 distinct database queries.
  - However, the join between the employees and computers should be retrievable with one database query with a LEFT JOIN.

N+1 problem: solutions

1. Refrain from eager loading in a relationship triggering the N+1 problem.
   - Set `fetch=FetchType.LAZY`.
   - This is usually a valid solution that removes the problem.
2. Tailor a JPQL query that implements the JOIN, and map the result to entities.
   - Now, a JPQL query produces a list of `Object[]` arrays, and eager loading is not triggered.
3. Specify a fetch group by means of an entity graph.
   - Fetch groups provide a way to define alternative search strategies for the same entities.
   - In each query, a strategy of choice can be applied.
   - For an example of usage, see e.g.
     - https://thoughts-on-java.org/jpa-21-entity-graph-part-1-named-entity/
     - https://thoughts-on-java.org/jpa-21-entity-graph-part-2-define/

Criteria API

- Criteria API provides a programmatic way to implement queries and updates.
- The operations are constructed by classes and methods in the jakarta.persistence.criteria package.
  - Note that Hibernate's native Criteria API (org.hibernate.Criteria) is deprecated and should not be used.
- In Criteria API, each operation is constructed by a sequence of Java statements.
  - Cf. JPQL, where a dedicated query language is used.

Example

```
public class TiliRepositoryCustomImpl implements TiliRepositoryCustom {

    @PersistenceContext  2 usages
    private EntityManager entityManager;

    @Override  2 usages  vesavvo
    public List<Tili> findBySaldoIsAtLeast(BigDecimal minimi) {
        CriteriaBuilder cb = entityManager.getCriteriaBuilder();
        CriteriaQuery<Tili> cq = cb.createQuery(Tili.class);
        Root<Tili> tili = cq.from(Tili.class);
        cq.select(tili).where(cb.greaterThanOrEqualTo(tili.<~>get("saldo"), minimi));
        return entityManager.createQuery(cq).getResultList();
    }
}
```

- The code above retrieves the accounts with a balance higher than the given amount.

Phases of a Criteria query

1. Create a CriteriaBuilder object.
2. Request a CriteriaQuery object.
3. Specify the root of the query.
4. Specify the CriteriaQuery object to define the desired operation, starting from the root.
5. Execute the query using EntityManager's getResultList() method.

Specifying the result

- In the previous example, `select()` and `where()` methods of the `CriteriaQuery` interface were used.
- The `CriteriaQuery` interface specifies methods to tailor the query to match the goal.
  - The method names resemble their SQL counterparts.
- In addition, the methods in the `CriteriaBuilder` interface can be used.
  - These methods are analogous with SQL operators and functions.

Batch updates and deletes

- In a batch query, the query object was created with createCriteriaQuery() method of the CriteriaBuilder class.
- Consequently:
  - createCriteriaUpdate() for batch updates
  - createCriteriaDelete() for batch deletes
- In both cases, the operation is executed via executeUpdate() instead of getResultList() that was used for search queries.

JPQL vs. Criteria API

JPQL

- An erroneous query produces a DB level error.
- SQL injection may be an issue.
- Easy to learn (analogous to SQL)

Java Criteria API

- An erroneous query is detected at compile time (as the code is written).
- Protection for SQL injections.
- May be challenging to learn.
- Better reusability.